

----- Importaciones

```
import pandas as pd
import numpy as np
from sklearn.preprocessing import MinMaxScaler, LabelEncoder
from sklearn.decomposition import PCA
```

----- Fin de las Importaciones

----- 2.se lee el archivo del proyecto para crear un data frame

```
EmpleadosAttrition = pd.read_csv(r"C:\\Users\\sator\\Desktop\\Tec de mty\\6.1\\EmpleadosAttrition.csv")
```

```
"""me sirve para poder ver una forma general del dataset, para su limpieza"""
EmpleadosAttrition.shape
```

```
(400, 30)
```

```
EmpleadosAttrition.head()
```

	Age	BusinessTravel	Department	DistanceFromHome	Education	EducationField
0	50	Travel_Rarely	Research & Development	1 km	2	Medical Devices
1	36	Travel_Rarely	Research & Development	6 km	2	Medical Devices
2	21	Travel_Rarely	Sales	7 km	1	Marketing
3	52	Travel_Rarely	Research & Development	7 km	4	Life Sciences
4	33	Travel_Rarely	Research & Development	15 km	1	Medical Devices

```
5 rows × 30 columns
```

```
EmpleadosAttrition.dtypes
```

```

Age                int64
BusinessTravel     object
Department         object
DistanceFromHome   object
Education          int64
EducationField     object
EmployeeCount      int64
EmployeeNumber     int64
EnvironmentSatisfaction  int64
Gender             object
JobInvolvement     int64
JobLevel           int64
JobRole            object
JobSatisfaction    int64
MaritalStatus      object
MonthlyIncome      int64
NumCompaniesWorked int64
HiringDate         object
Over18             object
OverTime           object
PercentSalaryHike  int64
PerformanceRating  int64
RelationshipSatisfaction  int64
StandardHours      int64
TotalWorkingYears  int64
TrainingTimesLastYear  int64
WorkLifeBalance    int64
YearsInCurrentRole int64
YearsSinceLastPromotion int64
Attrition          object
dtype: object

```

EmpleadosAttrition.info()

```

<class 'pandas.core.frame.DataFrame'>
RangeIndex: 400 entries, 0 to 399
Data columns (total 30 columns):
#   Column                                Non-Null Count  Dtype
---  -
0   Age                                   400 non-null    int64
1   BusinessTravel                       396 non-null    object
2   Department                           400 non-null    object
3   DistanceFromHome                     400 non-null    object
4   Education                             400 non-null    int64
5   EducationField                       400 non-null    object
6   EmployeeCount                        400 non-null    int64
7   EmployeeNumber                       400 non-null    int64
8   EnvironmentSatisfaction               400 non-null    int64
9   Gender                               400 non-null    object
10  JobInvolvement                       400 non-null    int64
11  JobLevel                             400 non-null    int64
12  JobRole                              400 non-null    object
13  JobSatisfaction                       400 non-null    int64
14  MaritalStatus                        395 non-null    object
15  MonthlyIncome                        400 non-null    int64
16  NumCompaniesWorked                   400 non-null    int64
17  HiringDate                           400 non-null    object
18  Over18                               400 non-null    object

```

```

19 OverTime          400 non-null    object
20 PercentSalaryHike 400 non-null    int64
21 PerformanceRating 400 non-null    int64
22 RelationshipSatisfaction 400 non-null    int64
23 StandardHours      400 non-null    int64
24 TotalWorkingYears  400 non-null    int64
25 TrainingTimesLastYear 400 non-null    int64
26 WorkLifeBalance    400 non-null    int64
27 YearsInCurrentRole  400 non-null    int64
28 YearsSinceLastPromotion 400 non-null    int64
29 Attrition          400 non-null    object
dtypes: int64(19), object(11)
memory usage: 93.9+ KB

```

----- 3. se eliminan las columnas irrelevantes del df

```

columnas_irrelevantes = ["EmployeeCount", "EmployeeNumber", "Over18", "Stand
EmpleadosAttrition = EmpleadosAttrition.drop(columns=columnas_irrelevantes)

""se confirma la eliminacion de las columnas que no se necesitan""
EmpleadosAttrition.shape

(400, 26)

```

----- 4. antigüedad "HiringDate" viene en

formato m/d/aaaa, no esta estandarizada, con y sin ceros a la izquierda.

```

fechas_invalidas = EmpleadosAttrition["HiringDate"][
    pd.to_datetime(EmpleadosAttrition["HiringDate"], format="%m/%d/%Y", errc
]

print(fechas_invalidas.tolist())#hay una fecha que dice un dia que no existe

['2/30/2012']

```

----- 5. Crea una columna llamada Year y obtén el año de contratación del empleado a partir de su fecha 'HiringDate'. No se te olvide que debe ser un entero.

```
EmpleadosAttrition["Year"] = (EmpleadosAttrition["HiringDate"].str.split("/")
```

----- 6. Crea una columna llamada

YearsAtCompany que contenga los años que el

- ✓ empleado lleva en la compañía hasta el año 2018. Para su cálculo, usa la variable Year que acabas de crear.

```
EmpleadosAttrition["YearsAtCompany"] = 2018 - EmpleadosAttrition["Year"]
EmpleadosAttrition[["HiringDate", "Year", "YearsAtCompany"]].head()
```

	HiringDate	Year	YearsAtCompany
0	06/06/2013	2013	5
1	12/25/2015	2015	3
2	2/14/2017	2017	1
3	7/29/2010	2010	8
4	10/07/2011	2011	7

----- 7. La DistanceFromHome está dada en

- ✓ kilómetros, pero tiene las letras “km” al final y así no puede ser entera.

```
EmpleadosAttrition = EmpleadosAttrition.rename(
    columns={"DistanceFromHome": "DistanceFromHome_km"}
)
```

----- 8 y 9. Renombra la variable

- ✓ DistanceFromHome a DistanceFromHome_km.

```
EmpleadosAttrition["DistanceFromHome"] = (EmpleadosAttrition["DistanceFromHome_km"]
      """esto me garantiza que se puede conservar una columna donde aparezca la ca
EmpleadosAttrition[["DistanceFromHome_km", "DistanceFromHome"]].head()
```

	DistanceFromHome_km	DistanceFromHome
0	1 km	1
1	6 km	6
2	7 km	7
3	7 km	7
4	15 km	15

----- 10. Borra las columnas Year, HiringDate

- ✓ y DistanceFromHome_km debido a que ya no son útiles.

```
EmpleadosAttrition = EmpleadosAttrition.drop(columns=["Year", "HiringDate",
EmpleadosAttrition.columns
```

```
Index(['Age', 'BusinessTravel', 'Department', 'Education', 'EducationField',
      'EnvironmentSatisfaction', 'Gender', 'JobInvolvement', 'JobLevel',
      'JobRole', 'JobSatisfaction', 'MaritalStatus', 'MonthlyIncome',
      'NumCompaniesWorked', 'OverTime', 'PercentSalaryHike',
      'PerformanceRating', 'RelationshipSatisfaction', 'TotalWorkingYears',
      'TrainingTimesLastYear', 'WorkLifeBalance', 'YearsInCurrentRole',
      'YearsSinceLastPromotion', 'Attrition', 'YearsAtCompany',
      'DistanceFromHome'],
      dtype='object')
```

----- 11. Aprovechando los ajustes que se están haciendo, la empresa desea saber si todos los departamentos tienen un ingreso promedio similar.

Genera una nuevo frame llamado

SueldoPromedioDepto que contenga el

- ✓ MonthlyIncome promedio por departamento de los empleados y colócalo en una variable llamada SueldoPromedio. Esta tabla solo es informativa, no la

vas a utilizar en el set de datos que estás construyendo.

```
SueldoPromedioDepto = (EmpleadosAttrition.groupby("Department")["MonthlyIncome"].agg("mean"))
```

	Department	SueldoPromedio
0	Human Resources	6239.888889
1	Research & Development	6804.149813
2	Sales	7188.250000

----- 12. La variable MonthlyIncome tiene un valor numérico muy grande comparada con las otras variables. Escala dicha variable para que tenga un valor entre 0 y 1.

```
scaler = MinMaxScaler()
EmpleadosAttrition["MonthlyIncome"] = scaler.fit_transform(EmpleadosAttrition["MonthlyIncome"].values.reshape(-1,))
EmpleadosAttrition["MonthlyIncome"].describe()
```

```
count    400.000000
mean      0.311196
std       0.258308
min       0.000000
25%       0.106939
50%       0.222711
75%       0.462745
max       1.000000
Name: MonthlyIncome, dtype: float64
```

----- 13. Todo parece indicar que las variables categóricas que quedan sí son importantes para obtener la variable de salida. Convierte todas las variables categóricas que quedan a numéricas:

```

"""overtime es una variable categorica pero anteriormente no hice ninguna co
variables_categoricas = [
    "BusinessTravel", "Department", "EducationField",
    "Gender", "JobRole", "MaritalStatus", "OverTime", "Attrition"
]

encoders = {}
for col in variables_categoricas:
    le = LabelEncoder()
    EmpleadosAttrition[col] = le.fit_transform(EmpleadosAttrition[col])
    encoders[col] = le
    print(col, "->", dict(zip(le.classes_, le.transform(le.classes_))))

```

```

BusinessTravel -> {'Non-Travel': np.int64(0), 'Travel_Frequently': np.int64(1)
Department -> {'Human Resources': np.int64(0), 'Research & Development': np.i
EducationField -> {'Human Resources': np.int64(0), 'Life Sciences': np.int64(
Gender -> {'Female': np.int64(0), 'Male': np.int64(1)}
JobRole -> {'Healthcare Representative': np.int64(0), 'Human Resources': np.i
MaritalStatus -> {'Divorced': np.int64(0), 'Married': np.int64(1), 'Single':
OverTime -> {'No': np.int64(0), 'Yes': np.int64(1)}
Attrition -> {'No': np.int64(0), 'Yes': np.int64(1)}

```

----- 14. Ahora debes hacer la evaluación de las variables para quedarte con las mejores. Calcula la correlación lineal de cada una de las variables con respecto al Attrition.

```

correlaciones = EmpleadosAttrition.corr()["Attrition"].sort_values(key=abs,
correlaciones

```

```

Attrition          1.000000
OverTime           0.324777
JobLevel           -0.214266
TotalWorkingYears -0.213329
Age                -0.212121
YearsInCurrentRole -0.203918
MonthlyIncome      -0.194936
MaritalStatus      0.187283
YearsAtCompany     -0.176001
JobInvolvement     -0.166785
JobSatisfaction    -0.164957
EnvironmentSatisfaction -0.124327
JobRole            0.078684
TrainingTimesLastYear -0.070884
YearsSinceLastPromotion -0.069000
PercentSalaryHike  -0.060880
BusinessTravel     0.060677
Education          -0.055531
Department         0.054236
DistanceFromHome   0.052732
EducationField     0.051184
RelationshipSatisfaction -0.030945
Gender             -0.028839
WorkLifeBalance    -0.021723
NumCompaniesWorked -0.009082
PerformanceRating  -0.006471
Name: Attrition, dtype: float64

```

----- 15. Selecciona solo aquellas variables que tengan una correlación mayor o igual a 0.1, dejándolas en otro frame llamado EmpleadosAttritionFinal. No olvides mantener la variable de salidaAttrition; esto es equivalente a borrar las que no cumplen con el límite.

```

limite = 0.1
variables_seleccionadas = correlaciones[correlaciones >= limite].index.tolist()

if "Attrition" not in variables_seleccionadas:
    variables_seleccionadas.append("Attrition")

EmpleadosAttritionFinal = EmpleadosAttrition[variables_seleccionadas].copy()
print(f"cantidad de variables seleccionadas ({len(variables_seleccionadas)})")
print(variables_seleccionadas)
EmpleadosAttritionFinal.shape

```

```
cantidad de variables seleccionadas (3):
['Attrition', 'OverTime', 'MaritalStatus']
(400, 3)
```

----- 16. Crea una nueva variable llamada EmpleadosAttritionPCA formada por los componentes principales del frame

EmpleadosAttritionFinal. Recuerda que el resultado del proceso PCA es un numpy array, por lo que, para hacer referencia a una columna, por ejemplo, la 0, puedes usar la instrucción EmpleadosAttritionPCA[:,0]).

```
pca = PCA()
EmpleadosAttritionPCA = pca.fit_transform(EmpleadosAttritionFinal)
type(EmpleadosAttritionPCA), EmpleadosAttritionPCA.shape

EmpleadosAttritionPCA[:, 0]
```

```
array([-1.08604212, -1.08604212,  1.02124215,  0.89606008,  0.07652856,
       -1.03970461, -0.09499102, -1.03970461,  1.88711118,  0.89606008,
       -0.09499102, -0.09499102,  1.06757966,  0.03019105, -0.09499102,
       -0.09499102, -0.09499102, -1.08604212,  0.89606008, -1.08604212,
       -1.08604212, -0.04865351,  0.89606008,  0.07652856, -0.09499102,
       -0.04865351,  1.06757966, -0.04865351, -1.08604212, -0.04865351,
       -0.09499102, -0.91452254, -0.09499102, -0.09499102,  0.89606008,
       -1.03970461, -0.04865351, -0.09499102, -1.08604212,  0.89606008,
       -0.09499102,  0.89606008, -0.09499102, -0.04865351,  0.07652856,
       -1.08604212, -1.08604212, -0.09499102, -0.04865351, -0.04865351,
       -0.09499102, -1.03970461, -0.09499102,  0.89606008,  0.89606008,
       -1.08604212,  0.07652856, -1.08604212,  0.94239759,  0.89606008,
       0.89606008, -1.08604212, -0.04865351, -0.09499102, -1.08604212,
       -0.04865351,  1.88711118, -1.08604212, -0.09499102, -0.09499102,
       0.89606008,  0.89606008, -0.09499102, -0.09499102, -1.08604212,
       1.02124215, -0.09499102,  0.89606008, -1.08604212,  2.05863076,
       -1.08604212,  1.88711118, -0.04865351,  0.89606008,  0.07652856,
       -0.09499102, -0.09499102, -1.08604212, -0.09499102, -1.08604212,
       0.89606008,  0.89606008, -0.09499102, -1.03970461, -0.09499102,
       -0.09499102, -0.04865351, -0.04865351, -1.08604212, -1.08604212,
       0.89606008,  0.07652856, -0.09499102,  0.94239759, -1.08604212,
       1.02124215, -1.08604212, -0.09499102,  0.89606008, -0.09499102,
       1.02124215, -0.09499102, -0.04865351, -1.08604212, -0.09499102,
       -0.04865351,  0.89606008, -0.04865351, -0.09499102,  1.88711118,
       -1.08604212,  0.94239759,  0.03019105, -0.09499102,  0.89606008,
       -0.09499102, -0.09499102, -0.09499102,  0.89606008,  0.03019105,
       0.89606008,  1.02124215,  0.89606008,  0.89606008,  0.89606008,
```

```
-0.09499102, 0.89606008, -0.96086005, -0.09499102, 0.89606008,
-1.08604212, -1.08604212, -1.08604212, 0.03019105, -0.09499102,
-0.09499102, -0.09499102, -1.03970461, 0.89606008, -1.08604212,
1.06757966, -1.08604212, 0.94239759, -0.04865351, -0.09499102,
-0.09499102, 1.06757966, -0.09499102, -0.09499102, 0.94239759,
-0.04865351, -0.09499102, -1.03970461, 0.07652856, -0.09499102,
-0.09499102, -0.04865351, -0.09499102, 0.94239759, 0.89606008,
-0.09499102, -1.08604212, -0.09499102, -1.03970461, 0.89606008,
0.89606008, -1.08604212, 0.03019105, 0.89606008, -0.04865351,
0.89606008, -1.03970461, -0.09499102, 0.89606008, -0.09499102,
-0.04865351, -0.04865351, 0.94239759, -0.04865351, -1.08604212,
-1.03970461, -0.09499102, -0.09499102, -1.08604212, -1.03970461,
0.89606008, 1.06757966, -0.04865351, 0.89606008, 0.94239759,
0.07652856, 0.89606008, -1.03970461, -1.08604212, 1.06757966,
1.06757966, -1.08604212, -0.09499102, -0.09499102, 0.94239759,
-0.04865351, -0.09499102, -0.09499102, 1.06757966, 0.03019105,
0.89606008, -0.09499102, -0.09499102, 0.89606008, -1.08604212,
-0.09499102, -0.09499102, -0.09499102, 0.89606008, 1.06757966,
-1.08604212, -0.09499102, -0.91452254, 0.89606008, -0.09499102,
1.02124215, -0.09499102, -1.08604212, -0.04865351, 0.89606008,
-0.09499102, -0.09499102, -0.09499102, 0.94239759, -0.09499102,
0.89606008, 1.06757966, -0.09499102, 0.89606008, -0.09499102,
0.89606008, 0.89606008, -1.08604212, 1.02124215, 0.89606008,
1.02124215, -0.04865351, -0.09499102, -1.08604212, -0.09499102,
-0.09499102, 0.89606008, -0.96086005, -0.04865351, -0.09499102,
-0.09499102, 0.89606008, -0.04865351, 0.89606008, -0.09499102,
-1.08604212, -0.09499102, -0.09499102, -0.09499102, -1.08604212,
-1.03970461, -0.09499102, -0.09499102, -0.09499102, -0.09499102,
-1.08604212, -0.09499102, -0.96086005, -1.08604212, -0.09499102,
-0.09499102, -1.08604212, -0.09499102, 0.94239759, -1.03970461,
-0.09499102, 0.07652856, -0.09499102, -1.08604212, 1.02124215
```

----- 17. Agrega el mínimo número de Componentes Principales en columnas del frame EmpleadosAttritionPCA que logren explicar el 80% de la varianza, al frame EmpleadosAttritionFinal. Puedes usar la instrucción assign, columna por columna, llamando a cada una C0, C1, etc., hasta las que vayas a agregar.

```
varianza_acumulada = np.cumsum(pca.explained_variance_ratio_)
n_componentes = int(np.argmax(varianza_acumulada >= 0.80) + 1)
print(f"componentes necesarios para explicar el 80% de la varianza: {n_compo
print(f"varianza acumulada con esos componentes: {varianza_acumulada[n_compo

for indice_componente in range(n_componentes):
    EmpleadosAttritionFinal = EmpleadosAttritionFinal.assign(**{f"C{indice_c
```

```
EmpleadosAttritionFinal.head()
```

componentes necesarios para explicar el 80% de la varianza: 2
varianza acumulada con esos componentes: 0.8856

	Attrition	OverTime	MaritalStatus		C0	C1
0	0	0	0		-1.086042	-0.204715
1	0	0	0		-1.086042	-0.204715
2	1	0	2		1.021242	0.071437
3	0	0	2		0.896060	-0.407184
4	1	1	1		0.076529	1.044837

----- 18. Guarda el set de datos que has formado y que tienes en EmpleadosAttritionFinal en un archivo CSV llamado EmpleadosAttritionFinal.csv.

- ✓ Las últimas columnas que colocaste quedarán después de la variable Attrition, lo cual no importa, pero si gustas lo puedes arreglar antes de escribir el archivo.

```
columnas_pca = [f"C{indice_componente}" for indice_componente in range(n_corr)]
col_ord = [
    col for col in EmpleadosAttritionFinal.columns
    if col not in columnas_pca + ["Attrition"]] + columnas_pca + ["Attrition"]

EmpleadosAttritionFinal = EmpleadosAttritionFinal[col_ord]
EmpleadosAttritionFinal.to_csv("EmpleadosAttritionFinal.csv", index=False)

EmpleadosAttritionFinal.head()
```

OverTime	MaritalStatus	C0	C1	Attrition
----------	---------------	----	----	-----------